

BitBull

System Architecture & Implementation Reference

A prediction-market exchange for live 5-minute BTC binary markets, built as an application layer around GameBull's real matching engine, plus a standalone read-only perpetual-futures hedging sidecar.

Primary repo	~/Desktop/gb-crypto-local (app + drivers, GameBull-integrated)
Hedging sidecar	~/gb-crypto-hedging-service (standalone, read-only, TypeScript/Fastify)
Boundary	GameBull's matching engine, trading-api, and distribution engine are never modified — read/targeted-d
Document scope	Everything implemented through the current session: pricing model, AMM ladder, risk gating, settlement

Generated as an internal engineering reference. Not a spec for future work — a map of what exists today, why it's built that way, and what's known to still be unresolved.

Contents

- 1** System Overview & Design Principles
- 2** Repository & Process Map
- 3** Data Flow: A Trade End to End
- 4** Pricing Model (Three Layers)
- 5** The AMM Liquidity Ladder
- 6** Risk Gating & Inventory Control
- 7** Directional / Expiry Lockout Behavior
- 8** Order Types, Matching & Settlement
- 9** Fees, Realized Spread & P&L; Attribution
- 10** Frontend
- 11** The Hedging Service
- 12** Calibration & Diagnostic Metrics
- 13** Issues Found & Fixed (Chronological)
- 14** Known Unresolved Issues & Structural Limits
- 15** Explored but Not Implemented
- 16** Reference: Key Configuration Constants

1. System Overview & Design Principles

BitBull lets users trade live 5-minute BTC binary markets ("Will BTCUSDT be $\geq$ \$X at time T?") against a house market-making bot and each other. Every fill routes through GameBull's real **trading-api** → **matcher** → **distribution engine** — BitBull is an application layer built around that engine, not a replacement for it.

1.1 The non-negotiable boundary

Never modified

GameBull's matching engine (gb-trading-matching-engine-service), trading-api, and distribution engine are treated as fixed infrastructure. Every interaction with them is either a normal API call (placing bids exactly as any client would) or a read/targeted-delete against their own DynamoDB/MySQL tables, mirroring patterns the app already uses (e.g. cancelling the house's own stale resting rows). No schema change, no matcher logic change, ever.

1.2 Three price layers, never collapsed into one

The system deliberately keeps three distinct notions of "price" separate, because collapsing them was the single biggest early design flaw (a fair-value estimate was originally shown to users as if it were an executable price):

Layer	What it is	Used by
1. Theoretical probability	Black-Scholes-style fair value from spot, strike, vol, time (digitalProbYes)	House quoting only — never shown to users as a tradable price
2. Liquidity quotes	House bid/ask resting in the CLOB, sized by the AMM	The order book itself — what can actually be matched
3. Market price discovery	bestBid / bestAsk / marketPrice / lastTrade, derived from the live book	What users see and transact against

Users never transact at the fair-value number or at a naive "mid" — a YES buy always executes against the real bestAsk, a NO buy against the real bestBid's complement.

1.3 Design principles that recur throughout the system

- **Inventory adjusts price in exactly one place** (quoting.mjs's reservation-price logic). The risk engine (risk.mjs) only gates or sizes — it never re-prices.
- **Event-driven requoting, not a blind timer.** Quotes refresh instantly on any house-touching fill, on a fair-value-movement change detector (~8s), and a 60s unconditional safety net in case the detector itself silently fails.
- **P&L prioritizes realized values.** netPnL sums only realized components; the theoretical quoted-spread estimate is tracked separately as an efficiency metric, never counted as profit.
- **The gamma wall is treated as one recurring problem, fixed the same way everywhere it appears** — time-decaying liquidity depth, a time-varying hedge deadband, an expiry lockout, and (later) a directional-lockout mode all key off the same $\tau_{\blacksquare} = \text{clamp}(\tau/\text{REF_SEC}, 0, 1)$ clock.

2. Repository & Process Map

2.1 Two independent codebases

Repo	Role	Language
~/Desktop/gb-crypto-local	The exchange app itself: web server, AMM/pricing drivers, settlement, frontend. Integrated directly with GameBull's services.	Node.js (ESM), vanilla JS/HTML frontend
~/gb-crypto-hedging-service	Standalone perpetual-futures hedging sidecar. Reads BitBull's house inventory from Redis (read-only) and manages an offsetting BTC perp position.	TypeScript, Fastify

The two communicate only through Redis (BitBull publishes inventory, the hedging service reads it) and a small HTTP control plane (the app can call the hedging service's /reset on a round reset). Neither imports the other's code.

2.2 gb-crypto-local process map

Process / file	Runs as	Responsibility
app/server.mjs	Long-running HTTP server (:5050)	Routes, trade() order handling, settlement orchestration, mmDesk() P&L, lifecycle loop, event-driven requote triggers
drivers/mmp-pricing/index.mjs	Spawned fresh per quote cycle (not a daemon)	Computes fair value, reservation price/spread, AMM ladder depth, risk gates, posts house bids
drivers/market-generator/index.mjs	Spawned when the rolling market needs replacing	Opens the next 5-minute market at an ATM-rounded strike
drivers/settlement/index.mjs	Spawned by settle()	Resolves an expired market via GameBull's real distribution chain
drivers/oracle-feed/index.mjs	Long-running (own Binance WS)	Publishes live BTCUSDT spot to Redis (CRYPTO_SPOT_BTCUSDT)
drivers/inventory-mirror/index.mjs	Long-running, --watch mode	Publishes house YES/NO inventory to Redis for the hedging service to read
app/public/index.html	Static, served by server.mjs	Full frontend: market list, detail view, ticket, portfolio, Hedge Desk

2.3 gb-crypto-hedging-service process map

Module	Responsibility
src/index.ts	Bootstrap: wires config → inventory source → feed → venue → gate → hedger → loop → control plane
src/inventory/gamebull.ts	Reads BitBull's house inventory from Redis, computes aggregate BTC delta across active markets
src/feed/binance-ws.ts	The service's own independent Binance spot WebSocket (not shared with BitBull's oracle-feed)
src/core/gate.ts	Vol/notional gate deciding whether the hedge is "armed" this tick
src/core/hedger.ts	Reconciles the live venue position to the target delta; tracks fees, slippage, realized/unrealized hedge P&L
src/core/taper.ts	linearTaper() — the time-varying hedge deadband
src/loop.ts	The ~4s control loop: read spot → poll inventory → gate → reconcile hedge → record to ledger
src/venue/binance-demo.ts / dry-run.ts	Execution venues — real Binance USDT-M futures demo/testnet, or a fully simulated fill engine
src/http/server.ts	Control plane: GET /health /state /ledger /report, POST /reset /config /kill

Safety rail baked into config.ts

`assertPaper()` hard-blocks any FUTURES_BASE pointed at a real Binance production host (api.binance.com, fapi.binance.com, etc.) — the service refuses to start rather than risk touching real money. This check runs even in dry-run mode.

3. Data Flow: A Trade End to End

Market open

- lifecycle() (server.mjs, every 8s) finds no live market → runs market-generator (strike = spot, ATM-rounded) → runs mmp-pricing to seed both sides.

A user places a market order

- Frontend openTicket() → renderTicket() (market-only UI) → POST /api/trade { orderType:'market' }.
- server.mjs's trade(): reads the live order book, walks the OPPOSITE side's resting rows via walkBook() in the matcher's own price-priority order (ORDER BY bid_amount DESC, row_id) to find real executable depth, caps the submitted quantity to that depth (IOC-intent, not atomic exchange-level IOC), then POSTs to GameBull's trading-api exactly like any real client.
- After the fill: settleFill() (fees.mjs) re-derives the actual matched quantity/price, splits the fill into house-attributed vs. user-attributed shares via the same walkBook() logic, and updates fee/spread/cost-basis Redis keys.
- If the fill touched house liquidity: an instant, un-awaited re-quote is fired for that market.

Continuous background loops (server.mjs)

- lifecycle() every 8s — settle any expired market, open the next one.
- Fair-value-movement check every 8s — re-quotes only markets whose fair value moved ≥ 1 probability point since the last posted quote (a real trigger condition, not a blind timer).
- Unconditional safety net every 60s — re-quotes regardless, purely to guard against the fair-value check itself silently failing.

Settlement

- settle(marketId): forces the market's expiry into the past, runs GameBull's real settlement/distribution driver chain (assign winning → distribute winning), then reads back every matched bid to accrue realized P&L (house-side into app:mm:settledPnl, per-user into app:realized) and credit user wallets.

Hedging (separate process)

- gb-crypto-hedging-service's loop ticks independently every ~4s: reads its own spot feed, polls house inventory from Redis, computes aggregate BTC delta, checks the gate, and reconciles the venue position toward that delta — entirely decoupled from BitBull's own request/response cycle.

4. Pricing Model (drivers/lib/pricing.mjs, quoting.mjs)

4.1 Fair value — digitalProbYes

A digital (binary) option's fair probability under GBM with zero drift:

```
d = (ln(spot/strike) - 0.5*sigma^2*tau) / (sigma*sqrt(tau))
p(YES) = N(d)           # standard normal CDF
dp/dS = normPdf(d) / (spot * sigma * sqrt(tau))  # the digital's delta
```

Used only as an input to house quoting — never displayed to users as a tradable price (Layer 1 of the three-layer model).

4.2 Reservation price & dynamic spread (quoting.mjs)

An Avellaneda-Stoikov-style heuristic, ported from a prior simulation project as an inventory-control shape (not a validated exact pricing model). The reservation price shifts fair value based on the house's own inventory skew:

```
netSkew = qYes - qNo    (= houseNo - houseYes, inventory-mirror publishes with a documente
d swap)
reservation = clamp(pFair + (netSkew/b)*gamma*sigma^2*tauHat, 0.03, 0.97)

spread = gamma*sigma^2*tauHat
        + (2/gamma)*ln(1+gamma/k)           # AS inventory-depth term
        + pinRisk + invWidenTerm + flowWidenTerm
half = clamp(spread/2, 0.01, 0.03)          # keeps the quoted market 2-6c wide
bid = clamp(reservation-half, 0.03, 0.97)
ask = clamp(reservation+half, 0.03, 0.97)
```

Too-long-YES inventory ($\text{houseYes} > \text{houseNo}$) $\rightarrow$ $\text{netSkew} < 0 \rightarrow$ reservation drops below fair value, discouraging further YES accumulation — verified numerically. **This is the only place inventory is allowed to adjust price**; the risk engine downstream only gates or sizes.

The pin-risk term (gamma-wall fix, part 1)

$\text{pinRisk} = \text{gammaWiden} * \text{pFair} * (1 - \text{pFair}) / \text{sqrt}(\text{tauHat})$ — a binary's gamma is proportional to $p(1-p)$ (peaks exactly ATM) and diverges as $\tau \rightarrow 0$. This term widens the spread precisely where and when that divergence is worst.

4.3 LMSR-derived ladder depth (not a flat size)

Instead of a constant N-shares-per-level, each rung's size is exactly what an LMSR market maker with liquidity parameter b would itself quote to move its own price between two ticks:

```
p(q) = 1 / (1 + exp(-(qYes-qNo)/b))  =>  qYes-qNo = b * logit(p)
lmsrStepQty(pFrom, pTo, b) = b * |logit(pTo) - logit(pFrom)|
```

This is smallest near $p=0.5$ (a few shares move the price a full cent) and grows in the tails — a genuine LMSR property, not a UI choice, and the opposite of what a naive risk-reducing market maker would want in the tails on its own (handled separately by the risk gate, Section 6).

4.4 Time-decaying liquidity parameter (ammBForTau)

```
ammBForTau(tauSec, bMax, bMin):
    tauHat = clamp(tauSec / REF_SEC, 0, 1)    # REF_SEC = 300 (5-minute tenor)
    return bMin + (bMax - bMin) * tauHat
```

Full depth (bMax) most of the market's life — when there's still time for inventory to mean-revert or the hedge to catch up — shrinking toward bMin (sized off a stated worst-case settlement-loss tolerance: $\text{loss} = b \cdot \ln(2)$) as the expiry lockout approaches. Uses the exact same τ clock as the pin-risk term, so spread-widening and depth-shrinking move together rather than being two uncoordinated mechanisms.

Constant	Current value	Meaning
AMM_B_MAX	25,000	Full-depth liquidity parameter, early in a market's life
AMM_B_MIN	1,500	Near-expiry floor — caps worst-case settlement loss at ~\$1,040 ($b \cdot \ln 2$)
LADDER_LEVELS	12	How far in price the ladder reaches (12c from touch at 1c steps)
LADDER_STEP	1c	Price spacing between ladder rungs
LADDER_MAX_QTY	3,000/level	Per-level cap (LMSR depth is otherwise unbounded near 0/100)

5. The AMM Liquidity Ladder (drivers/mmp-pricing/index.mjs)

Each quote cycle, for one market at a time (the app runs exactly one live 5-minute market at a time by design — never two overlapping), mmp-pricing:

- Cancels the house's own still-resting quotes on both sides (cancelRestingRows, generalized in pricing.mjs).
- Reads spot + market meta (strike, expiry) and computes fair value + tau.
- Reads current house inventory (qYes/qNo from inventory-mirror's Redis keys) and order-flow imbalance.
- Computes reservation price + dynamic spread (quoting.mjs), then the touch price for each side.
- Computes progressive quote-size (quoteSize) and a hard gate (withinLimit) per side against INV_LIMIT.
- Computes the time-decayed b (ammBForTau), then builds the ladder: 12 levels, each rung's size from lmsrStepQty at that b, price walking away from the touch by 1c per level.
- Posts every non-zero rung to GameBull's matcher via the exact same placeBid() a real client would use.

Event-driven full-ladder requoting (verified live)

On any house-touching fill, cancelRestingRows wipes the **entire** resting ladder on the affected side, not just the consumed levels — the whole ladder is rebuilt fresh from the new reservation price next cycle. Verified with real before/after order-book snapshots: a single large market-order sweep moves the touch price multiple ticks and removes untouched levels too, not just what was filled.

Ladder replenishment after a partial fill

If 500 of a level's resting shares are consumed and the underlying fair value hasn't moved, the next quote cycle rebuilds that same level back to its full LMSR-implied size (not a residual top-up) — because mmp-pricing recomputes the whole ladder from current inventory + fair value every cycle rather than patching. A 1,000-share sweep that fills across two price levels causes both levels to be fully rebuilt on the next cycle, not independently replenished to 500 each mid-sweep.

6. Risk Gating & Inventory Control (drivers/lib/risk.mjs)

Two pure functions, neither of which computes or modifies price — sizing/gating only, strictly downstream of quoting.mjs.

```
withinLimit(houseYes, houseNo, side, limit):
  return exposure(side) < limit           # final hard boolean backstop

quoteSize(baseQty, houseYes, houseNo, side, limit):
  room = max(0, limit - exposure(side))
  return max(0, min(baseQty, room))       # progressive taper, the PRIMARY lever
```

quoteSize provides a linear taper as inventory approaches the limit (confirmed live: 100% → 83% → 33% → 17% → 3% → 0% of base size as exposure climbs) — deliberately avoiding a sudden on/off cliff. withinLimit is the absolute final backstop for whatever quoteSize doesn't already zero out.

Important nuance surfaced this session

LMSR's raw math gets *deeper/stickier* in the tails as price approaches 0/100 — the opposite of what a risk-reducing market maker wants. The "fewer contracts as inventory gets lopsided" behavior the system actually exhibits comes entirely from this risk-gate taper, not from any LMSR property.

Constant	Current value	Meaning
INV_LIMIT	12,500 shares	Max one-sided house inventory per market. Raised 5,000 → 12,500 to fit the 12-level ladder's full depth.

7. Directional / Expiry Lockout Behavior

Inside the last EXPIRY_LOCKOUT_SEC (20s) before a market settles, the naive behavior would be to go fully dark on both sides (gamma-wall fix, part 2). This was extended mid-session into a more deliberate, directional behavior once the underlying mechanics were worked out correctly — and worked out incorrectly twice before landing on the right version (see Section 13 for the corrections).

7.1 The corrected mechanics

Key fact this whole feature depends on

server.mjs's bestBidAsk() is asymmetric: bestBid comes from the resting **YES** book; bestAsk = 100 - bestNoBid comes from the resting **NO** book. So a resting YES bid is what BOTH keeps YES structurally "no liquidity" for a direct buyer (no NO-side bid is ever posted) AND is what makes NO cheap for a buyer (NO price = 100 - bestBid). Posting a bid on the wrong side does the reverse of both — this was the actual bug in the first implementation attempt.

7.2 The final logic

- If $|\text{spot} - \text{strike}| < \20 (DIRECTIONAL_LOCKOUT_THRESHOLD_USD): genuinely uncertain even this close to expiry → full lockout, both sides dark, unchanged from the original design.
- If $|\text{spot} - \text{strike}| \geq \20 : the market has clearly resolved one way. Keep bidding the FAVORED side (the side statistically likely to pay out) — that's the side the house should end up holding, and its aggressive price is what makes the underdog side cheap for a risk-taker to buy.
- The favored side's price is computed directly off fair probability, bypassing the normal [0.03, 0.97] house-quote safety clamp (that clamp protects against adverse selection this scenario doesn't have) — but capped at AGGRESSIVE_CAP_CENTS.

7.3 The risk:reward cap (AGGRESSIVE_CAP_CENTS)

Pricing exactly at fair value always has $EV \approx 0$ by construction, so fair pricing alone never says when to stop. What actually gets worse as the favored price climbs toward 100c is risk *concentration*: at price p , the house risks p to win only $(100-p)$ — a ratio that explodes toward infinity as $p \rightarrow 100$.

$$\text{cap} = R * 100 / (R + 1) \quad \text{where } R = \text{max acceptable risk:reward ratio}$$

$R = 19 \Rightarrow \text{cap} = 95c$ (don't risk more than 19c for every 1c of potential gain)

Past the cap, the favored bid is held flat at 95c rather than chasing fair value further — the underdog's cheapest achievable price via this mechanism is then floored at 5c (100-95), not 1-2c, even when true fair value implies more. Verified live: fair value implied a 100c YES bid, correctly capped at 95c, with the underdog correctly showing bestAsk:null / tradeable at 5c.

8. Order Types, Matching & Settlement

8.1 Market orders (IOC-intent)

The only order type exposed in the current UI (the Market/Limit toggle was removed for simplicity; the backend still supports `orderType:'limit'` end-to-end). A market order:

- Reads the live opposite-side book and walks it via `walkBook()` (`drivers/lib/matching.mjs`) in the matcher's own price-priority order (ORDER BY `bid_amount` DESC, `row_id` ASC) — empirically verified against the real matcher, not assumed from reading its source alone.
- Caps the submitted quantity to actual executable depth — "IOC-intent", explicitly documented as application-layer depth-capping plus post-trade residual cleanup, not a claim of atomic exchange-level IOC (the matcher itself is external and out of scope to change).
- Submits at an aggressive limit price (worst swept price + a small slippage buffer) so a quote refresh landing mid-flight never leaves the order short-filled — costs a little extra wallet reservation, never a worse fill price (the matcher still prices each fill off the resting bid it actually crosses).

8.2 `walkBook()` — shared by two callers

One implementation serves both market-order depth-capping and fee-engine counterparty attribution (`settleFill`) — no duplicated price-priority logic.

8.3 Settlement (`settle()`, reusing GameBull's real chain)

- Forces the market's stored expiry into the past.
- Runs the real settlement driver, then GameBull's own `/pt/assign/winning` and `/pt/distribute/winning` endpoints — exactly the production distribution chain, not a simulated shortcut.
- Reads back every matched bid for that market, computes stake/win/pnl per bid, accrues realized P&L (house into `app:mm:settledPnl`, per-user into `app:realized`), and credits winner wallets via the wallet-stub's batch-process endpoint.

Units bug, fixed this session (see Section 13)

`bb_users.unused_amount` / `bb_pending_bids.winningAmount` are both in **points** (100 points = \$1). `settle()` originally converted to dollars before crediting the wallet, which itself expects points — a 100x underpayment on every settlement. Fixed by passing `winPoints` directly. Historical underpaid settlements from before the fix were **not** retroactively corrected (flagged as an open, undecided item — Section 14).

9. Fees, Realized Spread & P&L Attribution

9.1 settleFill() (drivers/lib/fees.mjs)

A single post-fill step computes fees, realized spread, and cost-basis together, since all three need the same actual-fill data. Counterparty attribution reuses walkBook() to split a fill into house-attributed vs. user-attributed shares.

```
computeFee(shares, execPriceCents):
  p = execPriceCents / 100
  return shares * p * (1-p) * 0.07      # user-user fills only; house trades are fee-free
```

Field	Computed from
userShares / feeRevenue	Rows where the counterparty's userId != MMP_USER_ID (a real user-user fill). Currently structurally \$0 — see Section 14.
houseShares / realizedSpreadRevenue	fairValueAtFill vs. actual execution price, on the house-attributed portion of the fill only
Cost basis (app:mm:invCost:{marketId})	Incremental running yesQty/yesCostSum, noQty/noCostSum — updated ONLY at confirmed-fill time, per market

9.2 The six P&L components (mmDesk())

Component	Meaning	In netPnL?
quotedSpreadRevenue	Theoretical estimate from house-matched volume * HALF_SPREAD	No — efficiency metric only
realizedSpreadRevenue	Actual spread captured per-fill, fair value vs. execution	Yes
feeRevenue	User-user taker fees from actual fills	Yes
inventoryPnl	Unrealized mark-to-market on currently-active house inventory only	Yes
hedgePnl	External hedge layer P&L (from the hedging service's /state)	Yes
settlementPnl	Realized P&L from final settlement	Yes

```
netPnl = realizedSpreadRevenue + feeRevenue + inventoryPnl + hedgePnl + settlementPnl - fees
```

A market's P&L is counted in exactly one bucket at a time: while active it contributes to the live inventoryPnl sum; the instant settle() resolves it, that contribution is credited into settlementPnl and its per-market cost-basis key simply stops being summed — inventory-mirror already prunes the corresponding Redis keys for markets no longer active, so the handoff happens atomically with no double-count or drop.

10. Frontend (app/public/index.html)

10.1 Market cards

- Show marketPrice / lastTrade (Layer 3) — never fair value (Layer 1), keeping the three-layer separation on the UI too.
- hasLiquidity(m, side) checks bestAsk!=null (YES) / bestBid!=null (NO); a side with none greys out with 'No liquidity' text instead of a stale/misleading price.
- A defense-in-depth guard in openTicket() re-checks liquidity at click time and toasts a warning instead of opening a ticket for a side that went dark in the ~1-2s between poll and click.

10.2 Detail page

- Live BTC-vs-strike chart, strike/spot distance readout, order book panel (resting YES/NO rows), YES-probability readout.
- BUY YES / BUY NO buttons mirror the same hasLiquidity/disabled logic as the card grid — verified live catching the market naturally inside its own expiry-lockout window, both buttons correctly greyed with 'No liquidity'.

10.3 Portfolio, wallet, Hedge Desk

- Portfolio splits open positions from settled history (won/lost, frozen realized P&L, capped at 12 recent).
- Wallet statement modal reads bb_wallet_txns (every debit/credit with a running balance), private per-user.
- Hedge Desk tab renders all six P&L components plus the four calibration metrics as distinct stat tiles, plus a live activity log and leaderboard.

10.4 Bugs fixed on the frontend (see Section 13 for full detail)

- Countdown flicker (two competing tick sources fighting over the same DOM node).
- Detail view staying visible after navigating away via the nav bar (only the hash-based exit path cleaned up state).
- Net MM P&L stat tile overflowing its box (CSS Grid's implicit min-width:auto blocking shrink).
- Contracts/max-payout not live-updating on quantity change (missing element ids for the renderer to target).
- Ngrok serving a stale build (no cache headers at all — browsers heuristically cached per-origin).

11. The Hedging Service (gb-crypto-hedging-service)

A standalone, read-only perpetual-futures hedging sidecar. It never writes to BitBull's data, never touches GameBull, and refuses to start against a production exchange host.

11.1 The control loop (loop.ts, ~4s tick)

```
spot = getSpot() # own independent Binance WS feed
sigmaPerSec = max(realizedVol, minSigmaPerSec)
inv = inventory.poll(spot, sigmaPerSec) # aggregate BTC delta across active markets
gate = gate.update(vol, inv.notionalUsdt, hedgerEnabled)
markPrice = venue.getMarkPrice() # REAL perp mark, not spot (see 11.3)
deadbandUsdt = linearTaper(nearestTauSec, refSec, tight, loose)
target = gate.armed ? inv.agggregatedDelta : 0
hedger.reconcile(target, markPrice, deadbandUsdt)
```

11.2 Gamma-wall guards on the inventory side (inventory/gamebull.ts)

- expiryLockoutSec (20s): markets inside their own lockout window are skipped entirely for delta purposes — their position resolves itself, chasing a diverging delta only loses fees on a hedge with no time left to pay off.
- minTauSec (60s floor): tau is floored at this value for the delta calculation only (never for the lockout check) — past this point dp/dS is a mathematical fiction, so the exposure is deliberately under-reported rather than demanding an unexecutable hedge.

11.3 The spot-vs-perp basis fix

Originally, spot was fed into `hedger.reconcile()/flatten()` as the `markPrice` parameter for everything downstream — notional cap, deadband threshold, fees, unrealized P&L marking. But real execution happens on the perp (binance-demo hits Binance's real USDT-M futures API), which trades at a funding-driven basis to spot.

Measured live basis (2026-07-24)

Spot \$65,425.09 vs. perp mark \$65,395.30 — a \$29.79 / ~4.5bps gap. The real ledger showed ~5.6bps average slippagePaid across historical fills, consistent with this. Fixed by using the venue's own `getMarkPrice()` (real perp mark for binance-demo, unchanged spot-mirrored value for dry-run) for sizing/notional/fee/P&L math, while keeping spot for the digital-delta calculation (correctly tied to what the market's payoff actually depends on).

11.4 Time-varying hedge deadband (taper.ts)

```
linearTaper(tauSec, refSec, early, late):
  tauHat = clamp(tauSec/refSec, 0, 1)
  return late + (early - late) * tauHat
```

Tight (\$125) far from expiry — gamma is small there, a real signal is cheap to react to. Loosening toward \$400 as the nearest active market's tau shrinks toward 300s — where chasing small moves is mostly fee churn on a position

about to resolve itself. Mirrors ammBForTau's shape, just in the opposite direction (grows instead of shrinks toward expiry). Confirmed live with the deadband value visibly climbing as an actual market aged.

11.5 Leverage & liquidation (analysis, not yet acted on)

Current config: leverage=1, multiAssets=true (cross-margin, USDT+USDC+other assets pooled). Real demo-account balance checked live: ~\$9,400 available cross-margined. At 1x, MAX_NOTIONAL_USDT=\$10,000 is already close to the practical ceiling this capital supports. Leverage doesn't change directional exposure (delta is delta regardless), only how much margin a given notional requires — raising leverage to 5-10x would support \$47k-\$94k of notional with the same capital, with a liquidation buffer estimated at roughly 10-20%+ adverse move (cross-margined against the whole account, not isolated per-position margin). Not yet implemented.

12. Calibration & Diagnostic Metrics

Metric	Definition	Live reading (session snapshot)
quoteFillRate	fillCount / quoteCount — of all house quotes posted, what fraction filled	0.001 (very low — expected with thin test volume)
adverseSelection	Fair-value movement AGAINST the house 10s after each house fill; positive=bad, per-fill sample, 200-sample rolling window	-0.32c avg on 4 samples — too few to trust the sign yet
inventoryTurnover	houseMatched volume / average houseYes-houseNo — a first-pass proxy for recycling speed, flagged for refinement	0 (no turnover observed yet)
spreadCaptureEfficiency	realizedSpreadRevenue / quotedSpreadRevenue — how much of the theoretical spread was actually captured	0.674

All four are genuinely live and wired (Redis-backed, updated on real fills), but sample sizes so far are small — most of this session's trading volume is a single test account, not organic multi-user flow. Treat current readings as "the pipe works," not as trustworthy calibration numbers yet.

13. Issues Found & Fixed (Chronological)

Every bug below was found through live investigation against the running system, not inferred from reading code alone — the recurring pattern this session was to pull real numbers, force a live reproduction where possible, and verify the fix the same way.

Ghost zero-quantity orders corrupting bestBidAsk()

What happened: YES side showed a phantom 1c price with no real liquidity behind it.

Root cause: A fully-consumed order left `current_bid_count=0`, `is_matched=0` in MySQL — not flagged matched, so it leaked into resting-order calculations.

Fix: Added `WHERE current_bid_count > 0` to the `orderBook()` query, mirroring the matcher's own candidate-query semantics.

100x wallet-underpayment on settlement

What happened: Users were credited 100x less than they actually won.

Root cause: `settle()` converted points to dollars before crediting the wallet stub, which itself expects points.

Fix: Pass `winPoints` directly. Historical underpaid settlements were NOT retroactively corrected — flagged as an open, undecided item.

Countdown flicker on market cards

What happened: The countdown timer visibly stuttered every second.

Root cause: Two code paths (a local smooth per-second tick, and `refresh()`'s server-cached `tauSec`) both wrote the same DOM text node, fighting each other.

Fix: `refresh()` now only updates `data-expiry`, never the countdown text directly.

Detail view stuck after navigating away

What happened: Clicking Markets/Portfolio while viewing a market detail left the detail panel visibly stuck on screen.

Root cause: `tab()` never hid `tab-detail` or cleared `DETAIL/timers` — only the hash-based `exitDetail()` path did.

Fix: `tab()` now performs the same cleanup itself (clear `DETAIL`, disconnect the detail WS, clear the order-book timer, clear the hash without triggering a re-entrant hashchange).

Net MM P&L stat tile overflowing its box

What happened: A large P&L number visually broke out of its tile.

Root cause: CSS Grid's default min-width:auto prevented the grid item from shrinking below its unbreakable content width.

Fix: Added min-width:0 + overflow-wrap:break-word, reduced the stat font size slightly.

24.7-hour-stale oracle spot price

What happened: The live BTC chart showed prices wildly wrong vs. reality — jumping/oscillating near the strike when spot was actually far below it.

Root cause: The machine slept overnight; oracle-feed's WebSocket died silently on wake, and a self-disabling watchdog bug (checked `if (!this.lastTs) return``, with lastTs zeroed before a reconnect that itself never delivered a first message) meant the watchdog never fired again to notice.

Fix: Added connectAt tracking so the watchdog's staleness check has a valid reference point even before the first message arrives. Identical fix later found and applied independently in the hedging service's own binance-ws.ts — same bug class, two separate feeds.

Gamma-wall unhedgeable exposure

What happened: The hedging service's inventory calc produced absurd BTC delta (60+ BTC for a ~4,000-share book) at very low tau.

Root cause: digitalDelta used raw, unfloored tau all the way to zero — a mathematical singularity at tau=0, not a real hedge requirement.

Fix: Added minTauSec (floor) and expiryLockoutSec (skip) guards, with exported defaults to avoid NaN-from-missing-config when the new options were initially made non-optional and broke two existing tests.

Hedge sizing/execution basis error

What happened: Hedge orders were sized, deadband-gated, and fee/P&L-marked using spot, but actually filled on the perp.

Root cause: loop.ts fed ``spot`` into hedger.reconcile()/flatten() as the markPrice parameter for everything, never querying the venue's real mark price.

Fix: Use venue.getMarkPrice() (real perp mark) downstream of the delta calc, keep spot only for the delta itself. Verified live: measured basis \$29.79 / ~4.5bps, consistent with the ledger's own ~5.6bps historical average slippagePaid.

Directional-lockout mechanics inverted (found twice, in one session)

What happened: First implementation skipped the FAVORED side and bid the underdog directly — the opposite of the intended effect.

Root cause: Misread which resting book each of bestBid/bestAsk actually derives from (bestBid from the YES book, bestAsk from the NO book, not symmetric). A second attempted fix (flip to a direct underdog bid past a risk cap) repeated the same error in the other direction.

Fix: Re-derived the mechanics directly from server.mjs's bestBidAsk() source rather than from memory, then implemented: keep bidding the favored side (capped at 95c via the risk:reward derivation), never post a direct bid on the underdog. Verified live via forced strike/expiry overrides on a real market, both before and after the risk-reward cap change.

Dynamic delta-aware inventory cap live-tested as unusably restrictive

What happened: Implemented, syntax-checked, and immediately live-tested a cap deriving `INV_LIMIT` from `MAX_NOTIONAL_USDT / (dpdS * spot)` each cycle.

Root cause: Not a bug in the formula — it correctly revealed that a \$10k (or even a leveraged \$94k) hedge budget is drastically undersized relative to a 5-minute ATM BTC binary's real gamma, even under completely ordinary conditions (139s left, near-ATM), not just the extreme final-seconds case it targeted. The cap collapsed to 13-180 shares under normal operation.

Status: Defaulted OFF (`MMP_DYNAMIC_INV_CAP=1` to opt in) immediately after the live test, per an explicit rollback requirement given before implementation. Code path is fully wired and ready; needs a real design decision (a floor on the cap, scoping it only near the existing lockout window, or accepting the mismatch) before defaulting on.

14. Known Unresolved Issues & Structural Limits

Hedge budget structurally undersized vs. AMM inventory limit

What happened: INV_LIMIT (12,500 shares) and MAX_NOTIONAL_USDT (hedging service's \$10,000 hedge budget) were sized independently and never reconciled.

Root cause: Worst-case ATM delta at INV_LIMIT implies up to \$16M of real hedge notional at the tau floor — ~1,600x beyond the \$10k budget. Real ledger data showed exposure windows of \$300k-\$550k against a \$10k cap during two of Sid's real fills.

Status: Directly implicated in both of Sid's real settlement losses this session (\$719 and \$351). The dynamic-cap fix was attempted and rolled back (see above) — still open, needs a real design decision.

Fee revenue is structurally \$0

What happened: feeRevenue has never gone above \$0 despite real trading volume.

Root cause: settleFill() only counts a fill as fee-eligible when the counterparty's userId != MMP_USER_ID (matched against another user's RESTING order). The frontend hardcodes orderType:'market' with the Market/Limit toggle removed — no user can ever rest liquidity for another user to hit, so every fill's counterparty is structurally the house.

Status: Backend already supports orderType:'limit' end-to-end — re-exposing the toggle in the UI would fix this without new backend work. Not yet decided/implemented.

One whale can dominate a market's entire risk budget

What happened: A single test account (Sid) generated both of this session's real settlement losses via two moderately-sized trades.

Root cause: No per-user or per-market position cap exists. Researched Kalshi's real regulated equivalent (\$25,000/market retail default) as a design precedent.

Status: Proposed but not implemented: a \$1,500 per-user-per-market notional cap (~10-15% of INV_LIMIT's dollar-equivalent ceiling). Note: this specific cap would NOT have prevented either of the two actual historical losses observed (both were already under \$1,500) — it protects against a larger, not-yet-seen failure mode.

Sub-cent pricing near expiry blocked architecturally

What happened: Requested feature: let quotes approach 0.1c/0.01c granularity in the closing seconds to capture pure noise profit.

Root cause: GameBull's own trading-api validates bidAmount % inputPriceInterval == 0, and inputPriceInterval is fixed per-market at creation time (not changeable mid-market). Would require a market-wide tick change plus deliberately relaxing the locked-in [0.03,0.97] house-quote safety clamp.

Status: Explicitly declined without the operator's direct go-ahead, given the safety-clamp relaxation involved — still open, deliberately not implemented.

Historical settlement underpayment not backfilled

What happened: The 100x wallet-underpayment bug (Section 13) paid out real historical settlements 100x too low before the fix landed.

Root cause: The code fix does not retroactively correct past underpaid markets.

Status: Offered as an optional reconciliation pass; remains undone and undecided.

15. Explored but Not Implemented

15.1 Options-based gamma hedging

A BTC perp only hedges delta (a perp's own delta is always exactly 1, no curvature) — it structurally cannot offset gamma. Real listed BTC options could hedge the digital's terminal gamma the perp can't, but the known blocker is tenor/strike mismatch: no listed venue offers 5-minute granularity, so this would hedge the *aggregate* gamma of a whole book of short-dated markets (e.g. via a daily straddle), not any single market's individual terminal spike.

15.2 Cascading multi-tenor hedging (5m → 15m → 1hr → 24hr)

gamebull.ts's aggregateDelta already sums delta across every concurrently active market — but BitBull only ever runs one market at a time by design, so there's currently nothing to net against. Running multiple tenors concurrently (a real architecture change) would let this netting happen automatically with no new build for the netting logic itself, but netting only helps when the book happens to be skewed in opposite directions across tenors — not guaranteed, and doesn't address any single market's own terminal gamma spike.

15.3 Polymarket-style minting as an external hedge

Polymarket runs a real 5-minute BTC Up/Down market (confirmed live, launched Feb 2026, Chainlink-settled). Its conditional-token mint mechanic (\$1 collateral → 1 YES + 1 NO, independently redeemable, no need to hold both) would let the house acquire exactly the hedge leg it needs (mint, keep the wanted side, sell the unwanted side) without ever holding a naked position. Complications identified: Polymarket's own market has an independently-set strike/expiry (not identical to BitBull's own, so this is a correlated-not-perfect hedge, the same basis-risk shape as the spot-vs-perp problem in a different form); thinner liquidity than Binance perps; full (\$1/contract) collateral required vs. a perp's leveraged margin; a separate, independent expiry clock (a second gamma-wall risk stacked on the first); non-instant redemption (oracle dispute window); not suited to the continuous rebalancing a perp hedge needs. Framed as a candidate supplementary/coarser hedge, not a perp replacement. Not implemented — would also require touching GameBull's matcher to replicate internally, which is out of scope; using real external Polymarket was the more viable framing discussed.

15.4 Leverage on the perp hedge

See Section 11.5 — analyzed with real account-balance numbers, a concrete recommendation floated (5-10x), not yet implemented.

16. Reference: Key Configuration Constants

gb-crypto-local / drivers/mmp-pricing/index.mjs

Constant	Default	Meaning
MMP_TICK	1	Price grid, cents (matches the market's inputPriceInterval)
MMP_LADDER_LEVELS	12	How many rungs the ladder posts
MMP_LADDER_STEP	1	Cents between rungs
MMP_AMM_B_MAX	25000	Full-depth LMSR liquidity parameter
MMP_AMM_B_MIN	1500	Near-expiry floor, sized off settlement-loss tolerance
MMP_LADDER_MAX_QTY	3000	Per-level cap
MMP_LADDER_MIN_QTY	25	Legacy flat fallback if a level rounds to 0
MMP_INV_LIMIT	12500	Max one-sided house inventory per market
MMP_EXPIRY_LOCKOUT	20	Seconds before expiry the lockout begins
MMP_DIRECTIONAL_LOCKOUT_USD	20	spot-strike threshold for directional (vs. full) lockout
MMP_MAX_RISK_REWARD	19	Risk:reward cap -> 95c favored-side price ceiling
MMP_DYNAMIC_INV_CAP	unset (off)	Opt-in flag for the delta-aware inventory cap; set =1 to enable
MMP_HEDGE_MAX_NOTIONAL_USDT	10000	Mirrors the hedging service's own budget, for the dynamic cap calc
MMP_GAMMA / MMP_SIGMA_P / MMP_K	1.0 / 0.10 / 100	Avellaneda-Stoikov reservation/spread shape parameters

gb-crypto-hedging-service / .env

Constant	Default	Meaning
HEDGE_INTERVAL_SEC	4	Control-loop tick interval
HEDGE_DEADBAND_USDT	125	Tight deadband, far from expiry
HEDGE_DEADBAND_LOOSE_USDT	400	Loose deadband, near expiry
HEDGE_DEADBAND_REF_SEC	300	Taper reference window (5-minute tenor)
HEDGE_MIN_TAU_SEC	60	Tau floor for the delta calc
HEDGE_EXPIRY_LOCKOUT_SEC	20	Markets skipped entirely inside this window
MAX_NOTIONAL_USDT	10000	Hard cap on hedge notional
HEDGE_LEVERAGE	1	Perp leverage setting
MULTI_ASSETS_MARGIN	true	Cross-margin across USDT+USDC+other assets
EXECUTION_VENUE	dry-run (currently: binance-demo live)	dry-run (simulated) or binance-demo (real testnet perp)

End of document.